



kubernetes



**The
package manager
for Kubernetes**



Overview

Oramai conosciamo le sfide che le applicazioni moderne comportano.

I «Team» devono affrontare un'enorme pressione per garantire che le applicazioni siano leggere e scalabili.

Le applicazioni devono essere altamente disponibili e in grado di sopportare carichi variabili.

Storicamente, le applicazioni sono state più comunemente implementate come monoliti o applicazioni di grandi dimensioni a livello singolo servite su aunico sistema.

Con il passare del tempo, il settore si è spostato verso un approccio a microservizi e/o verso piccole applicazioni multilivello servite su più sistemi (spesso distribuite utilizzando tecnologia a container).

Il settore ha iniziato a sfruttare strumenti come Kubernetes per orchestrare e scalare i propri microservizi containerizzati.

Kubernetes, tuttavia, presenta una serie di sfide.

Sebbene sia un efficace strumento di orchestrazione dei container, presenta una curva di apprendimento ripida che può essere difficile da superare per i team.

Uno strumento che aiuta a semplificare le sfide dell'esecuzione dei carichi di lavoro su Kubernetes è Helm.

Helm consente agli utenti di distribuire e gestire in modo più semplice il ciclo di vita delle applicazioni Kubernetes.

Astrae molte delle complessità alla base della configurazione delle applicazioni Kubernetes e consente ai team di essere più produttivi sulla piattaforma.

The background is a dark blue field filled with a complex pattern of concentric circles and radial lines, creating a tunnel-like or orbital effect. The lines are lighter blue and have a glowing, slightly blurred appearance, giving a sense of depth and motion. The overall aesthetic is high-tech and futuristic.

Approcci nella gestione delle risorse

Per distribuire un'applicazione su Kubernetes, dobbiamo interagire con l'API Kubernetes per creare risorse.

kubectl è lo strumento che utilizziamo per parlare con l'API Kubernetes.

kubectl è uno strumento di interfaccia a riga di comando (CLI) utilizzato per astrarre la complessità dell'API Kubernetes dagli utenti finali, consentendo loro di lavorare in modo più efficiente sulla piattaforma.

Discutiamo di come kubectl può essere utilizzato per gestire le risorse Kubernetes.

Lo strumento kubectl fornisce una serie di sottocomandi per creare e modificare le risorse in modo imperativo.

Quello che segue è un piccolo elenco di questi comandi:

- **Creare**
- **Descrivere**
- **Modificare**
- **Elimina**

I comandi kubectl seguono un formato comune:

kubectl <verbo> <sostantivo> <argomenti>

Il verbo si riferisce a uno dei sottocomandi kubectl e il sostantivo si riferisce a una particolare risorsa Kubernetes.

Ad esempio, è possibile eseguire il seguente comando per creare una distribuzione:

kubectl crea distribuzione my-deployment --image=busybox

Ciò istruirebbe kubectl a parlare con l'API di distribuzione e creare una nuova distribuzione denominata my-deployment, utilizzando l'immagine di busybox da Docker Hub.

Puoi usare kubectl per ottenere maggiori informazioni sulla distribuzione che è stata creata utilizzando il sottocomando di descrizione:

kubectl describe il deployment my-deployment

Questo comando recupera le informazioni sulla distribuzione e formatta il risultato in un formato leggibile che consente agli sviluppatori di ispezionare la distribuzione my-deployment in tempo reale su Kubernetes.

Imperative e declarative configuration

Se si desiderava una modifica alla distribuzione, uno sviluppatore potrebbe utilizzare il sottocomando di modifica per modificarlo sul posto:

kubectl delete deployment my-deployment

Questo comando aprirà un editor di testo, consentendoti di modificare la distribuzione.

Quando si tratta di eliminare la risorsa, l'utente può eseguire il sub-comando delete:

kubectl delete deployment my-deployment

Ciò indicherà all'API di eliminare la distribuzione denominata my-deployment.

Le risorse Kubernetes, una volta create, esistono nel cluster come file di risorse JSON, che possono essere esportati come file YAML per una maggiore leggibilità umana.

Sebbene possa essere più semplice creare risorse in modo imperativo utilizzando i sottocomandi kubectl, Kubernetes consente di gestire direttamente le risorse YAML in modo dichiarativo per ottenere un maggiore controllo sulla creazione delle risorse.

I sottocomandi kubectl non ti consentono sempre di configurare tutte le possibili opzioni di risorse, ma la creazione diretta dei file YAML ti consente di creare risorse in modo più flessibile e colmare le lacune che i sottocomandi kubectl possono contenere.

Quando si creano risorse in modo dichiarativo, gli utenti prima scrivono la risorsa che desiderano creare in formato YAML. Successivamente, utilizzano lo strumento kubectl per applicare la risorsa all'API Kubernetes.

Mentre nella configurazione imperativa gli sviluppatori utilizzano i sottocomandi kubectl per gestire le risorse, la configurazione dichiarativa si basa principalmente su un solo sottocomando: si applica.

kubectl apply -f my-deployment.yaml

Questo comando fornisce a Kubernetes una risorsa YAML che contiene una specifica di risorsa, sebbene sia possibile utilizzare anche il formato JSON.

Kubernetes deduce l'azione da eseguire sulle risorse (creare o modificare) in base alla loro esistenza o meno.

Un'applicazione può essere configurata in modo dichiarativo seguendo questi passaggi:

1. Innanzitutto, l'utente può creare un file chiamato deployment.yaml e fornire una specifica in formato YAML per la distribuzione.

2. La distribuzione può quindi essere creata con il seguente comando:

kubectl apply -f deployment.yaml

3. Se si desidera apportare una modifica alla distribuzione modificando ad esempio il numero di repliche, è necessario modificare il file deployment.yaml

4. Per applicare la modifica si rilancia il kubectl apply:

kubectl apply -f deployment.yaml

1. Innanzitutto, l'utente può creare un file chiamato deployment.yaml e fornire una specifica in formato YAML per la distribuzione.

2. La distribuzione può quindi essere creata con il seguente comando:

kubectl apply -f deployment.yaml

3. Se si desidera apportare una modifica alla distribuzione modificando ad esempio il numero di repliche, è necessario modificare il file deployment.yaml

4. Per applicare la modifica si rilancia il kubectl apply:

kubectl apply -f deployment.yaml

Dopo aver eseguito quel comando, Kubernetes applicherà la dichiarazione di distribuzione fornita sulla distribuzione applicata in precedenza.

5. Quando si tratta di eliminare un'applicazione, la documentazione di Kubernetes consiglia di farlo in modo imperativo; cioè, usando il sottocomando delete invece di apply:

kubectl delete -f deployment.yaml

6. Il sottocomando delete può essere reso più dichiarativo passando il flag -f e un nome di file. Ciò fornisce a kubectl il nome della risorsa da eliminare dichiarata in un file specifico e consente agli sviluppatori di continuare a gestire le risorse con file YAML dichiarativi.

Abbiamo spiegato come Kubernetes ha due diversi metodi di configurazione: imperativo e dichiarativo.

Una domanda da considerare è quali sono le sfide di cui gli utenti devono essere consapevoli quando creano risorse Kubernetes con metodologie imperative e dichiarative?

Analizziamo alcune delle sfide più comuni.

Ecco un breve elenco di risorse di cui uno sviluppatore dovrebbe essere a conoscenza:

- **Deployment**
- **StatefulSet**
- **Service**
- **Ingress**
- **ConfigMap**
- **Secret**
- **StorageClass**
- **PersistentVolumeClaim**
- **ServiceAccount**
- **Role**
- **RoleBinding**
- **Namespace**

Immediatamente, la distribuzione di un'applicazione su Kubernetes non è semplice come premere un grande pulsante rosso contrassegnato con Deploy.

Gli sviluppatori devono essere in grado di determinare quali risorse sono necessarie per distribuire la propria applicazione e devono comprendere tali risorse a un livello sufficientemente approfondito per poterle configurare in modo appropriato.

Ciò richiede molta conoscenza e formazione sulla piattaforma.

Sebbene la comprensione e la creazione di risorse possano già sembrare un grosso ostacolo, questo è in realtà solo l'inizio di molte diverse sfide operative.

Un metodo di configurazione delle risorse Kubernetes incoraggiato è mantenere la loro configurazione nel controllo del codice sorgente affinché i team possano modificarli e condividerli, il che consente anche al repository del controllo del codice sorgente di diventare la fonte della verità.

La configurazione definita nel controllo del codice sorgente (denominato "stato locale") viene quindi creata applicandole all'ambiente Kubernetes e le risorse diventano "attive" o entrano in quello che può essere chiamato "stato live".

Sembra abbastanza semplice, ma cosa succede quando gli sviluppatori devono apportare modifiche alle proprie risorse?

La risposta corretta sarebbe modificare i file locali e applicare le modifiche per sincronizzare lo stato locale con lo stato live nel tentativo di aggiornare la fonte della verità.

Tuttavia, questo non è ciò che di solito finisce per accadere.

Spesso è più semplice, a breve termine, modificare la risorsa live in atto con `kubectl patch` o `kubectl edit` e saltare completamente la modifica dei file locali.

Ciò si traduce in un'incoerenza di stato tra gli stati locali e attivi ed è un atto che rende difficile il ridimensionamento su Kubernetes.

I cicli di vita delle applicazioni sono difficili da gestire

La gestione del ciclo di vita è un termine complesso, ma in questo contesto lo chiameremo il concetto di installazione, aggiornamento e ripristino delle applicazioni.

Nel mondo Kubernetes, un'installazione creerebbe risorse per distribuire e configurare un'applicazione.

L'installazione iniziale creerebbe quella che qui chiamiamo versione 1 di un'applicazione.

Un aggiornamento, quindi, può essere pensato come una modifica o una modifica a una o più di queste risorse Kubernetes.

I cicli di vita delle applicazioni sono difficili da gestire

Ogni batch di modifiche può essere considerato come un singolo aggiornamento.

Uno sviluppatore potrebbe modificare una singola risorsa del servizio, portando il numero di versione alla versione 2.

Lo sviluppatore potrebbe quindi modificare una distribuzione, una mappa di configurazione e un servizio, portando il numero di versioni alla versione 3.

Man mano che le versioni più recenti di un'applicazione continuano a essere implementate su Kubernetes, diventa più difficile tenere traccia dei cambiamenti che si sono verificati.

I cicli di vita delle applicazioni sono difficili da gestire

Kubernetes, nella maggior parte dei casi, non ha un modo intrinseco di mantenere una cronologia delle modifiche.

Sebbene ciò renda più difficile tenere traccia degli aggiornamenti, rende anche molto più difficile il ripristino di una versione precedente di un'applicazione.

Supponiamo che uno sviluppatore abbia precedentemente apportato una modifica errata su una particolare risorsa.

Come fa un Team a sapere dove tornare indietro?

I cicli di vita delle applicazioni sono difficili da gestire

Il caso n-1 è particolarmente facile da risolvere, poiché è la versione più recente.

Cosa succede, tuttavia, se l'ultima versione stabile risale a cinque versioni fa?

I team spesso finiscono per cercare di risolvere i problemi perché non sono in grado di identificare rapidamente l'ultima configurazione stabile che funzionava in precedenza.

I Resource Files sono statici

Questa è una sfida che riguarda principalmente lo stile di configurazione dichiarativo dell'applicazione delle risorse YAML.

Parte della difficoltà nel seguire un approccio dichiarativo è che i file di risorse Kubernetes non sono progettati in modo nativo per essere parametrizzati.

I file di risorse sono in gran parte progettati per essere scritti per intero prima di essere applicati e il contenuto rimane la fonte della verità fino a quando il file non viene modificato.

Quando si ha a che fare con Kubernetes, questa può essere una realtà frustrante.

I Resource Files sono statici

Alcune risorse API possono essere lunghe, contenere molti campi personalizzabili diversi e può essere piuttosto complicato scrivere e configurare completamente le risorse YAML.

I file statici si prestano a diventare un modello standard (nel gergo Boilerplate).

Con «Boilerplate» si intende testo o codice che rimane in gran parte coerente in contesti diversi ma simili.

Questo diventa un problema se gli sviluppatori gestiscono più applicazioni diverse, dove potrebbero potenzialmente gestire più risorse di distribuzione diverse, più servizi diversi e così via.

I Resource Files sono statici

Confrontando i file di risorse delle diverse applicazioni, potresti trovare un gran numero di configurazioni YAML simili tra di loro. La figura seguente mostra un esempio di due risorse con una configurazione standard significativa tra di loro. Il testo blu indica le righe standard, mentre il testo rosso indica le righe univoche:

```
apiVersion: v1
kind: Deployment
metadata:
  name: my-k8s-app
spec:
  replicas: 1
  selector:
    matchLabels:
      app: my-k8s-app
  strategy:
    rollingUpdate:
      maxSurge: 25%
      maxUnavailable: 25%
  template:
    metadata:
      labels:
        app: my-k8s-app
    spec:
      containers:
        - image: my-k8s-app:v1
          imagePullPolicy: IfNotPresent
          name: app
```

```
apiVersion: v1
kind: Deployment
metadata:
  name: your-k8s-app
spec:
  replicas: 1
  selector:
    matchLabels:
      app: your-k8s-app
  strategy:
    rollingUpdate:
      maxSurge: 25%
      maxUnavailable: 25%
  template:
    metadata:
      labels:
        app: your-k8s-app
    spec:
      containers:
        - image: your-k8s-app:v3
          imagePullPolicy: IfNotPresent
          name: app
```

Nota che in questo esempio ogni file è quasi esattamente lo stesso.

Quando si gestiscono file simili a questo, il boilerplate diventa un grosso problema per i team che gestiscono le proprie applicazioni in modo dichiarativo.

Nel corso del tempo, la community Kubernetes ha scoperto che la creazione e la manutenzione delle risorse Kubernetes per distribuire le applicazioni è difficile.

Ciò ha portato allo sviluppo di uno strumento semplice ma potente che avrebbe consentito ai team di superare le sfide poste dalla distribuzione di applicazioni su Kubernetes.

Lo strumento che è stato creato si chiama Helm.

The background is a dark blue field filled with a complex pattern of concentric circles and a grid of lines, creating a sense of depth and technology. The text is centered in the upper half of the image.

HELM: Introduzione

Helm è uno strumento open source utilizzato per creare pacchetti e distribuire applicazioni su Kubernetes.

Viene spesso chiamato Gestore pacchetti Kubernetes per le sue somiglianze con qualsiasi altro gestore di pacchetti che potresti trovare sul tuo sistema operativo preferito.

Helm è ampiamente utilizzato nella comunità Kubernetes ed è un progetto graduato CNCF.

Date le somiglianze di Helm con i gestori di pacchetti tradizionali, iniziamo a esplorare Helm esaminando prima come funziona un gestore di pacchetti.

I gestori di pacchetti vengono utilizzati per semplificare il processo di installazione, aggiornamento, ripristino e rimozione delle applicazioni di sistema.

Queste applicazioni sono definite in unità, chiamate pacchetti, che contengono metadati relativi al software di destinazione e alle sue dipendenze.

Il processo alla base dei gestori di pacchetti è semplice. Innanzitutto, l'utente passa il nome di un pacchetto software come argomento.

Il gestore pacchetti quindi esegue una ricerca in un repository di pacchetti per vedere se quel pacchetto esiste.

Se viene trovato, il gestore pacchetti installa l'applicazione definita dal pacchetto e le sue dipendenze nelle ubicazioni specificate nel sistema.

I gestori di pacchetti rendono la gestione del software molto semplice.

Ad esempio, immaginiamo di voler installare htop, un monitor di sistema Linux, su una macchina Fedora.

L'installazione sarebbe semplice come digitare un singolo comando:

`dnf install htop --assumeyes`

Questo indica a dnf, il gestore di pacchetti Fedora dal 2015, di trovare htop nel repository dei pacchetti Fedora e installarlo.

dnf si occupa anche di installare le dipendenze del pacchetto htop, quindi non dovresti preoccuparti di installare i suoi requisiti in anticipo.

Dopo che dnf ha trovato il pacchetto htop dal repository upstream, ti chiede se sei sicuro di voler procedere.

Il flag `--assumeyes` risponde automaticamente sì a questa domanda ea qualsiasi altro prompt che `dnf` potrebbe chiedere.

Nel tempo, versioni più recenti di `htop` potrebbero apparire nel repository upstream. `dnf` e altri gestori di pacchetti consentono agli utenti di aggiornare in modo efficiente a nuove versioni del software.

Il sottocomando che consente agli utenti di eseguire l'aggiornamento utilizzando `dnf` è `upgrade`:

`dnf upgrade htop --assumeyes`

Questo è un processo potente perché il gestore di pacchetti consente agli utenti di eseguire rapidamente il rollback se viene segnalato un bug critico o una vulnerabilità.

Se vuoi rimuovere completamente un'applicazione, un gestore di pacchetti può occuparsi anche di questo. dnf fornisce il sottocomando remove per questo scopo:

dnf remove htop -assumeyes

In questa sezione, abbiamo esaminato come utilizzare il gestore di pacchetti dnf su Fedora per gestire un pacchetto software.

Helm, in quanto gestore di pacchetti Kubernetes, è simile a dnf, sia nello scopo che nelle funzionalità.

Mentre dnf viene utilizzato per gestire le applicazioni su Fedora, Helm viene utilizzato per gestire le applicazioni su Kubernetes.

Lo esploreremo più in dettaglio in seguito.

Dato che Helm è stato progettato per fornire un'esperienza simile a quella dei gestori di pacchetti, gli utenti esperti di dnf o strumenti simili comprenderanno immediatamente i concetti di base di Helm.

Le cose si complicano, tuttavia, quando si parla dei dettagli specifici dell'implementazione.

dnf opera su pacchetti RPM che forniscono eseguibili, informazioni sulle dipendenze e metadati.

Helm, d'altra parte, funziona con i «charts».

Un «chart» Helm può essere pensato come un pacchetto Kubernetes.

I charts contengono i file di risorse dichiarativi Kubernetes necessari per distribuire un'applicazione.

Simile a un RPM, può anche dichiarare una o più dipendenze di cui l'applicazione ha bisogno per l'esecuzione.

Helm si basa su repository per fornire un accesso diffuso ai charts.

Gli sviluppatori di charts creano file YAML dichiarativi, li impacchettano in chart e li pubblicano nei repository di charts.

Gli utenti finali utilizzano quindi Helm per cercare i charts esistenti da distribuire su Kubernetes, in modo simile a come gli utenti finali di dnf cercheranno i pacchetti RPM da distribuire su Fedora.

Passiamo attraverso un esempio di base: Helm può essere utilizzato per distribuire Redis (un sistema di cache in memory) su Kubernetes utilizzando un chart pubblicato in un repository upstream.

Questo potrebbe essere eseguito utilizzando il comando di installazione di Helm:

helm install redis bitnami/redis --namespace=redis

Ciò installerebbe il chart redis dal repository del grafico bitnami in uno spazio dei nomi Kubernetes chiamato redis.

Questa installazione viene definita revisione iniziale o distribuzione iniziale di un chart Helm.

Se diventa disponibile una nuova versione del chart redis, gli utenti possono eseguire l'aggiornamento a una nuova versione utilizzando il comando upgrade:

helm upgrade redis bitnami/redis --namespace=redis

Ciò aggiornerebbe Redis per soddisfare le specifiche definite dal nuovo chart redis-ha.

Con i sistemi operativi, gli utenti dovrebbero preoccuparsi dei rollback se viene rilevato un bug o una vulnerabilità.

La stessa preoccupazione esiste con le applicazioni su Kubernetes e Helm fornisce il comando rollback per gestire questo caso d'uso:

helm rollback redis 1 --namespace=redis

Questo comando riporterà Redis alla sua prima revisione

Infine, Helm offre la possibilità di rimuovere completamente Redis con il comando di disinstallazione:

helm uninstall redis --namespace=redis

Con una comprensione di come funziona Helm come gestore di pacchetti, discutiamo più in dettaglio i vantaggi che Helm porta a Kubernetes.

I vantaggi di Helm In precedenza in questo capitolo, abbiamo esaminato come vengono create le applicazioni Kubernetes gestendo le risorse Kubernetes e non abbiamo maledetto alcune delle sfide coinvolte.

Ecco alcuni modi in cui Helm può superare queste sfide.

Supponiamo che a uno sviluppatore sia stato assegnato il compito di distribuire un database MySQL su Kubernetes.

Lo sviluppatore dovrebbe creare le risorse necessarie per configurare i suoi contenitori, rete e storage.

La quantità di conoscenza di Kubernetes richiesta per configurare tale applicazione da zero è elevata ed è un grosso ostacolo da superare per gli utenti Kubernetes nuovi e anche intermedi.

Con Helm, uno sviluppatore incaricato di distribuire un database MySQL potrebbe semplicemente cercare charts MySQL nei repository di charts upstream.

Questi charts sarebbero già stati scritti dagli sviluppatori di charts nella comunità e conterrebbero già la configurazione dichiarativa necessaria per distribuire un database MySQL.

A questo proposito, gli sviluppatori con questo tipo di attività agirebbero come semplici utenti finali che utilizzano Helm in modo simile a qualsiasi altro gestore di pacchetti.

Helm ha un concetto chiamato cronologia delle versioni.

Quando un chart Helm viene installato per la prima volta, Helm aggiunge quella revisione iniziale alla cronologia.

La cronologia viene ulteriormente modificata man mano che le revisioni aumentano tramite gli aggiornamenti, mantenendo varie istantanee di come l'applicazione è stata configurata a diverse revisioni.

Il processo di tracciamento di ogni revisione offre opportunità di rollback.

I rollback in Helm sono molto semplici.

Gli utenti puntano semplicemente Helm a una revisione precedente e Helm ripristina lo stato live a quello della revisione selezionata.

Con Helm, sono finiti i giorni del backup n-1.

Helm consente agli utenti di ripristinare le proprie applicazioni quanto desiderano, anche alla prima installazione.

Uno dei maggiori problemi con la creazione di risorse in modo dichiarativo è che le risorse Kubernetes sono statiche e non possono essere parametrizzate.

Come ricorderete in precedenza, ciò si traduce in risorse che diventano «bolierplate» tra applicazioni e configurazioni simili, rendendo più difficile per i team configurare le proprie applicazioni come codice.

Helm allevia questi problemi introducendo valori e modelli.

I valori sono semplicemente ciò che Helm chiama parametri per i charts.

I modelli sono file generati dinamicamente in base a un determinato insieme di valori f.

Questi due costrutti forniscono agli sviluppatori di charts la possibilità di scrivere risorse kubernetes che vengono generate automaticamente in base ai valori forniti dagli utenti.

In tal modo, le applicazioni gestite da Helm diventano più flessibili, meno standardizzate e più facili da mantenere

I valori e i modelli consentono agli utenti di eseguire operazioni come le seguenti:

- **Parametrizzare i campi comuni, come il nome dell'immagine in un Deployment e le porte in un servizio.**
- **Genera lunghe parti della configurazione YAML in base all'input dell'utente, come i montaggi del volume in una distribuzione o i dati in una ConfigMap.**
- **Includere o escludere risorse in base all'input dell'utente**

La capacità di generare dinamicamente file di risorse dichiarativi semplifica la creazione di risorse basate su YAML, garantendo al tempo stesso che le applicazioni vengano create in modo facilmente riproducibile.

I gestori di pacchetti evitano che gli utenti debbano gestire manualmente un'applicazione e le sue dipendenze.

Tutta la gestione può essere eseguita tramite il gestore di pacchetti stesso.

La stessa idea vale per Helm.

Poiché un chart Helm contiene una configurazione flessibile delle risorse Kubernetes, gli utenti non dovrebbero dover apportare modifiche direttamente alle risorse Kubernetes attive.

Gli utenti che desiderano modificare le proprie applicazioni possono farlo fornendo nuovi valori a un chart Helm o aggiornando la propria applicazione a una versione più recente del chart associato.

Ciò consente allo stato locale (rappresentato dalla configurazione del chart Helm) e allo stato live di rimanere coerenti tra le modifiche, offrendo agli utenti la possibilità di fornire una fonte di verità per le configurazioni delle risorse Kubernetes.

Helm semplifica le distribuzioni delle applicazioni determinando l'ordine in cui devono essere create le risorse Kubernetes.

Helm analizza ciascuna delle risorse di un chart e le ordina in base ai loro tipi.

Questo ordine pre-deterministico esiste per garantire che le risorse che comunemente hanno risorse dipendenti da esse vengano create per prime. Ad esempio, i segreti e le mappe di configurazione dovrebbero essere creati prima delle distribuzioni, poiché una distribuzione probabilmente consumerebbe tali risorse come volumi.

Helm esegue questo ordinamento senza alcuna interazione da parte dell'utente, quindi questa complessità viene astratta e impedisce agli utenti di doversi preoccupare dell'ordine in cui vengono applicate queste risorse.

Simile ad altri gestori di pacchetti, Helm offre la possibilità di definire hook del ciclo di vita.

Gli hook del ciclo di vita sono azioni che si verificano automaticamente in diverse fasi del ciclo di vita di un'applicazione.

Possono essere usati per fare :

- **Eseguire un backup dei dati su un aggiornamento.**
- **Ripristina i dati su un rollback.**
- **Convalidare un ambiente Kubernetes prima dell'installazione.**

Gli hook del ciclo di vita sono preziosi perché astraggono le complessità intorno alle attività che potrebbero non essere specifiche di Kubernetes.

Ad esempio, un utente Kubernetes potrebbe non avere familiarità con le best practice alla base del backup di un database o potrebbe non sapere quando eseguire tale attività.

Gli hook del ciclo di vita consentono agli esperti di scrivere un'automazione che esegua le migliori pratiche quando consigliato, in modo che gli utenti possano continuare a essere produttivi senza doversi preoccupare di quei dettagli.



HELM

Helm è uno strumento con impostazioni predefinite ragionevoli che consentono agli utenti di essere produttivi senza dover eseguire un numero elevato di attività dopo l'installazione.

Detto questo, ci sono diverse opzioni che gli utenti possono modificare o abilitare per modificare il comportamento di Helm.

Tratteremo queste opzioni nelle slides seguenti, iniziando con la configurazione dei repository upstream.

Helm fornisce il sottocomando `repo` per consentire agli utenti di gestire i repository di charts pre-configurati, il quale, a sua volta, contiene sottocomandi aggiuntivi che possono essere utilizzati per eseguire azioni su repository specifici:

- **`add`: per aggiungere un repository di charts**
- **`list`: per elencare i repository di charts**
- **`remove`: per rimuovere un repository di charts**
- **`update`: per aggiornare le informazioni sulle carte disponibili localmente dai repository di charts**
- **`index`: per generare un file indice data una directory contenente charts pre-confezionati**

```
helm repo add $REPO_NAME $REPO_URL  
helm repo add https://charts.bitnami.com/bitnami  
helm repo list
```

Nel tempo, gli aggiornamenti ai charts verranno pubblicati e rilasciati in questi repository.

I metadati del repository vengono memorizzati nella cache localmente.

Di conseguenza, Helm non è automaticamente consapevole quando un chart viene aggiornato.

Puoi indicare a Helm di controllare gli aggiornamenti da ogni repository aggiunto eseguendo il sottocomando di aggiornamento del repository.

Una volta eseguito questo comando, sarai in grado di installare gli ultimi charts da ciascun repository:

helm repo update

Potrebbe anche essere necessario rimuovere i repository che sono stati aggiunti in precedenza.

Ciò può essere ottenuto utilizzando il sottocomando repo remove:

helm repo remove bitnami

L'ultimo modulo di sottocomando repo rimasto è index; viene utilizzato dai manutentori di repository e charts per pubblicare charts nuovi o aggiornati.

I plugin sono funzionalità aggiuntive che possono essere utilizzate per fornire funzionalità aggiuntive a Helm.

La maggior parte degli utenti non dovrà preoccuparsi dei plug-in e della gestione dei plug-in con Helm.

Helm è uno strumento potente di per sé ed è completo delle funzionalità che promette fuori dagli schemi.

Detto questo, la community di Helm mantiene una varietà di plugin diversi che possono essere utilizzati per migliorare le capacità di Helm.

Un elenco di questi plugin è disponibile su:

<https://helm.sh/docs/community/related/>

Helm fornisce un sottocomando plugin per la gestione dei plugin, che contiene ulteriori sottocomandi:

helm plugin install \$URL

helm plugin list

helm plugin uninstall \$PLUGIN

helm plugin update \$PLUGIN

Di seguito sono riportati alcuni esempi di plugin upstream:

- **helm diff:** esegue una differenza tra un rilascio distribuito e un aggiornamento di Helm proposto
- **helm secrets:** utilizzato per aiutare a nascondere i segreti dalle carte dell'elmo
- **helm monitor:** Utilizzato per monitorare un rilascio ed eseguire un roll-back se si verificano determinati eventi
- **helm unittest:** utilizzato per eseguire il test dell'unità su un grafico Helm

Continueremo a discutere le opzioni di configurazione di Helm esaminando le diverse variabili di ambiente che possono essere impostate per modificare vari aspetti del comportamento di Helm.

Helm si basa sull'esistenza di variabili esternalizzate per configurare le opzioni di basso livello. La documentazione di Helm elenca sei variabili di ambiente principali utilizzate per configurare Helm:

- **XDG_CACHE_HOME:** imposta una posizione alternativa per l'archiviazione dei file memorizzati nella cache.
- **XDG_CONFIG_HOME:** imposta una posizione alternativa per l'archiviazione della configurazione del timone.
- **XDG_DATA_HOME:** imposta una posizione alternativa per la memorizzazione dei dati Helm .
- **HELM_DRIVER:** imposta il driver di archiviazione back-end.
- **HELM_NO_PLUGINS:** disabilita i plugin.
- **KUBECONFIG:** imposta un file di configurazione Kubernetes alternativo

Helm aderisce alla XDG Base Directory Specification, che è progettata per fornire un modo standardizzato di definire dove si trovano i diversi file sul filesystem di un sistema operativo.

Sulla base della specifica XDG, Helm crea automaticamente tre diverse directory predefinite su ciascun sistema operativo come richiesto:

O.S.	Cache Path	Configuration Path	Data Path
Windows	%TEMP%\helm	%APPDATA%\helm	%APPDATA%\helm
MacOS	\$HOME/Library/Caches/helm	\$HOME/Library/Preferences/helm	\$HOME/Library/helm
Linux	\$HOME/.cache/helm	\$HOME/.config/helm	\$HOME/.local/share/helm

Helm utilizza il percorso della cache per i charts scaricati dai repository.

I charts installati vengono memorizzati nella cache del computer locale per consentire un'installazione più rapida del chart la volta successiva che vi si farà riferimento.

Per aggiornare la cache, un utente può eseguire il comando `helm repo update`, che aggiornerà i metadati del repository con le informazioni più recenti disponibili, oltre a salvare il chart nella cache locale.

Il percorso di configurazione viene utilizzato per salvare le informazioni sul repository che sono state aggiunte eseguendo il comando `helm repo add`.

Quando viene installato un grafico che non è stato memorizzato nella cache, Helm utilizza il percorso di configurazione per cercare l'URL del repository del chart.

Helm utilizza quell'URL per capire dove risiede il chart per il download.

Il percorso dei dati viene utilizzato per memorizzare i plugin.

Quando un plug-in viene installato utilizzando il comando `helm plugin install`, i dati del plug-in vengono archiviati in questa posizione.

Per quanto riguarda le restanti variabili di ambiente che abbiamo descritto in precedenza, `HELM_DRIVER` viene utilizzato per determinare come viene archiviato lo stato di rilascio in Kubernetes.

Il valore predefinito è `segreto`, che è anche il valore consigliato. `Secret` codificherà lo stato in Base64 in un Kubernetes Secret.

Altre opzioni sono configmap, che memorizzerà lo stato in una ConfigMap Kubernetes in chiaro e la memoria, che memorizzerà lo stato nella memoria del processo locale.

L'uso della memoria locale è destinato a scopi di test e non è adatto per scopi generici o ambienti di produzione.

La variabile di ambiente HELM_NO_PLUGINS viene utilizzata per disabilitare i plugin. Se non è impostato, il valore predefinito che mantiene i plug-in abilitati è 0. Per disabilitare i plug-in, la variabile deve essere impostata su 1.

La variabile di ambiente KUBECONFIG viene utilizzata per impostare il file utilizzato per l'autenticazione nel cluster Kubernetes. S

e non è impostato, il valore predefinito sarà ~/.kube/config.

Nella maggior parte dei casi, gli utenti non dovranno modificare questo valore.

Gli utenti di Bash e Z shell possono abilitare il completamento con Tab per semplificare l'utilizzo di Helm. Ovviamente consente il completamento automatico dei comandi Helm quando si preme il tasto Tab, consentendo agli utenti di eseguire le attività più velocemente e aiutando a prevenire errori di input.

E' simile al comportamento predefinito della maggior parte dei moderni emulatori di terminale. Quando viene premuto il tasto Tab, i terminali cercano di indovinare quale deve essere l'argomento successivo osservando lo stato del comando e l'ambiente.

Il completamento della scheda può essere abilitato eseguendo il comando seguente:

source <(helm completion \$SHELL)

La variabile \$SHELL deve essere bash o zsh.

Nota che il completamento automatico esisterà solo nelle finestre del terminale che eseguono il comando precedente, quindi anche altre finestre dovranno eseguire questo comando per sperimentare la funzione di completamento automatico.

Helm deve essere in grado di autenticarsi con un cluster Kubernetes per distribuire e gestire le applicazioni.

Si autentica facendo riferimento a un file kubeconfig, che specifica diversi cluster Kubernetes e come eseguire l'autenticazione su di essi.

Quelli di voi che stanno usando Minikube quando seguono questo libro non avranno bisogno di configurare l'autenticazione, poiché Minikube configura automaticamente un file kubeconfig ogni volta che viene creato un nuovo cluster.

Se non si sta utilizzando Minikube, tuttavia, dovranno probabilmente creare un file kubeconfig o averne uno fornito, a seconda della distribuzione Kubernetes che si sta utilizzando.

Un file kubeconfig può essere creato sfruttando tre diversi comandi kubectl:

kubectl config set-cluster: Il comando set-cluster definirà una voce cluster nel file kubeconfig. Determina il nome host o l'indirizzo IP del cluster Kubernetes, insieme alla sua autorità di certificazione.

kubectl config set-credentials: Il comando set-credentials definirà il nome di un utente insieme al metodo di autenticazione e ai dettagli. Questo comando può configurare una coppia nome utente e password, un certificato client, un token di connessione o un provider di autenticazione per consentire a utenti e amministratori di specificare diversi metodi di autenticazione.

kubectl config set-context: Il comando `set-context` viene utilizzato per associare una credenziale a un cluster. Una volta stabilita un'associazione tra una credenziale e un cluster, l'utente sarà in grado di autenticarsi nel cluster specificato utilizzando il metodo di autenticazione della credenziale.

kubectl config view: Il comando `kubectl config view` può essere utilizzato per visualizzare il file `kubeconfig`. Nota come i cluster, i contesti e le stanze utente di `kubeconfig` corrispondono ai comandi descritti in precedenza, come mostrato.

Una volta che è presente un file `kubeconfig` valido, `Kubectl` e `Helm` saranno in grado di interagire con un cluster `Kubernetes`.

Sebbene l'autenticazione sia un mezzo per confermare l'identità, l'autorizzazione definisce le azioni che un utente autenticato può eseguire.

Kubernetes utilizza il controllo degli accessi basato sui ruoli (RBAC - role-based access control) per eseguire l'autorizzazione su Kubernetes.

RBAC è un sistema di progettazione di ruoli e privilegi che possono essere assegnati a un determinato utente o gruppo di utenti.

Le azioni che un utente può eseguire su Kubernetes dipendono dai ruoli assegnati all'utente.

Kubernetes offre molti ruoli diversi sulla piattaforma. Di seguito sono elencati tre ruoli comuni:

cluster-admin: consente a un utente di eseguire qualsiasi azione su qualsiasi risorsa in tutto il cluster

edit: consente a un utente di leggere e scrivere sulla maggior parte delle risorse all'interno di uno spazio dei nomi o di un raggruppamento logico di risorse Kubernetes

view: Impedisce a un utente di modificare le risorse esistenti e consente solo agli utenti di leggere le risorse all'interno di uno spazio dei nomi

Poiché Helm si autentica su Kubernetes utilizzando le credenziali definite nel file kubeconfig, a Helm viene assegnato lo stesso livello di accesso degli utenti definiti nel file.

Se l'accesso in modifica è abilitato, si può presumere che Helm disponga di autorizzazioni sufficienti per installare applicazioni, nella maggior parte dei casi.

Per l'accesso solo in visualizzazione, Helm non sarà in grado di installare applicazioni, poiché questo livello di accesso è di sola lettura.

Gli utenti che eseguono Minikube ricevono l'amministratore del cluster per impostazione predefinita dopo la creazione del cluster.

Anche se questa non sarebbe la migliore pratica in un ambiente di produzione, è accettabile per l'apprendimento e la sperimentazione.

Chi esegue Minikube non dovrà preoccuparsi di configurare l'autorizzazione.

Chi utilizza altri tipi di cluster Kubernetes che non sono Minikube dovranno assicurarsi che gli sia assegnato almeno il ruolo di modifica per poter distribuire la maggior parte delle applicazioni con Helm.

Questo può essere fatto chiedendo a un amministratore di eseguire il seguente comando:

kubectl create clusterrolebinding \$USER-edit --clusterrole=edit --user=\$USER